Problem statement:

We are writing an algorithm to determine times that are available and common amongst a group of individuals given their schedules, whether it be work or personal. The purpose of this is to find a time that all group members are available for a meeting, fitting into everyone's schedule, (starting time, duration, and ending time). This way, scheduling a meeting is hassle free and it avoids conflicts with group members' work and personal schedules.

Input:

1. List of lists of tuples in the format of 'HH:MM' (Busy_Schedules)
2. List of tuples in the format of 'HH:MM' (Working_Period)
3. Integer which represents the duration of the meeting in minutes (Duration_of_the_Meeting)

Outputs:

1. List of tuples in the format of 'HH:MM' (Available_Time_Slots)
   First helper function: convert the time to decimal

Constants/Assumptions:

1. All time inputs are in 24-hour military format (HH:MM)
2. All Busy_Schedule and Working_period times are given precisely down to the minute
3. Available slot is only provided if all group members are free, within the constraints of their Working_period

Let **n** be the number of members in the group
Let **k** be the average number of busy intervals per person
Let **s** be the total number of available time slots

Steps counts:

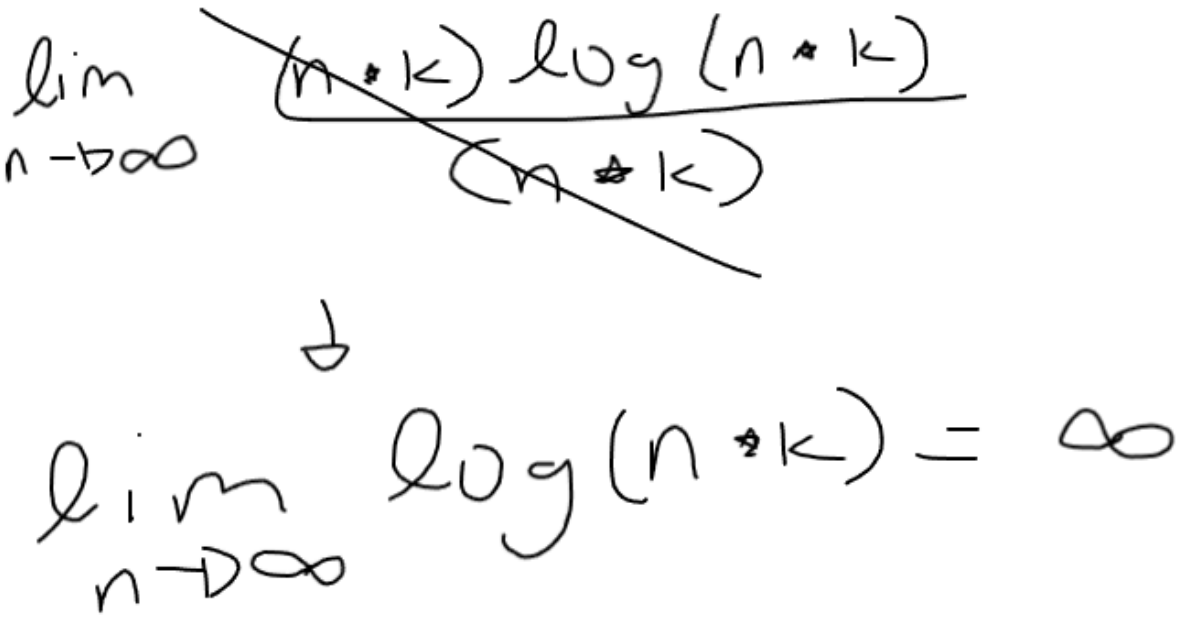
- Do: Converting the time tuples/strings to a decimal time
    1. Time complexity: O(n * k), O(1) - as each conversion is done in constant time. O(n * k) represents the total operations, as the conversion process is done for all time intervals
    2. Why?: We are putting the time into a format that can be handled mathematically by python, which is a necessary step
- Do: Assembling the unavailable_slots list
    1. Time complexity: O(n * k), as this operation loops through all the busy slots (k) of each member (n)
    2. Why?: We must make a list of the unavailable slots so that we can have data to compare against, utilizing it to find slots that are available.
- Do: Sorting the unavailable slots based on the start times

    1. Time complexity: O(m log m), where m can be rewritten as (n * k), because n (the number of members in the group) times k (the average number of busy intervals per person), is the total number of busy time intervals.
    2. Why?: This step sets up the next step, which is merging the overlapping unavailable slots.
- Do: Merge the overlapped unavailable slots
    1. Time complexity: O(n * k), as it iterates through the list
    2. Why?: When we combine the overlapped unavailable slots, it leaves less room for error, and it creates data that is smaller and more easily read, say, we have times 9:30-11:00 unavailable, and we also have 10:30-11:30 unavailable, this will be merged into 9:30-11:30 being unavailable. Avoiding sorting and merging will mean checks will have to be done for every unavailable time interval, which is more inefficient.
- Do: Compare the unavailable_slots with the working_period of each group member to identify gaps in which meetings can be scheduled
    1. Time complexity: O(n * k), O(k) - as each group member's working period just involves k number of unavailable_slots. With n amount of group members, the total time complexity is O(n * k).
    2. Why?: The whole purpose is to find the available time slots
- Do: Sort the available time slots in ascending order
    1. Time complexity: At a worst case scenario, time complexity is O(m log m), where m represents (n * k). Assuming s, the number of available time slots is less than n * k, which represents the total number of busy slots, sorting will be time complexity O(s log s).
    2. Why?: This is the final step in providing an output that is easily readable (in ascending order).

The Final Time Complexity and Efficiency Analysis

Time Complexity - O(m log m), where m represents (n * k), and the sorting step of the busy times dominates. The sorting step of the available time slots in ascending order *can* be equal to the time complexity of the busy times, but typically, it will not.

Proof by finding the Limit:



This proves that O(n*k log (n * k)) dominates O(n * k) as the input size increases. Making this step the most computationally expensive step.

But then why do it?:

Why sort the unavailable times slots and set them up to merge? Well, because without sorting, we will need to compare each interval with each and every other interval to check for overlaps, which would require O((n * k) ^2) operations, which is mathematically more complex than O (m log m) where m represents (n * k). A natural wonder is then, why even merge the unavailable times, since sorting is so taxing, and we are all doing it in order to merge. Is it really the most efficient? Merging the overlapped time intervals is the most efficient because it is literally meant to reduce complexity. Without merging, the algorithm needs to check all pairs of time intervals to see if they overlap or leave available gaps for each and every person, which increases the complexity even more, when time intervals could've just been simplified from the beginning.